

Progetto Informatica con Laboratorio

Equazione del calore 2D e riordinamento nelle fattorizzazioni Cholesky

Docenti:

Roberto Grossi Leonardo Robol

Studentessa:

Giulia Casti

Anno Accademico 2025/2026

1 Introduzione

Il presente documento descrive lo sviluppo e l'evoluzione software del modulo `HeatEquationSolver`, progettato per la risoluzione numerica dell'equazione del calore 2D e lo studio del riordinamento delle matrici sparse nelle fattorizzazioni di Cholesky.

2 Modulo: `HeatEquationSolver`

Lo sviluppo del solutore procede in modo modulare ed evolutivo. Di seguito vengono dettagliati i singoli sottomoduli.

2.1 Sottomodulo 1: Generazione della griglia del dominio

2.1.1 Specifiche e Funzionalità

Il componente si occupa della discretizzazione spaziale del dominio di calcolo.

- **Input:** Intero N che determina la discretizzazione uniforme del dominio quadrato $[0, 1]^2$.
- **Output:** Griglia 2D rappresentata come insieme di punti.

La griglia è definita come un insieme di punti dove ogni elemento è identificato da una coppia di coordinate (x, y) . La generazione include tutti i punti del dominio, compreso il bordo, con passo spaziale $h = \frac{1}{N}$, per un totale di $(N + 1) \times (N + 1)$ punti.

2.1.2 Strutture Dati

Per la rappresentazione in memoria delle entità coinvolte si adottano le seguenti scelte architetturali:

- **Punto:** Struttura (`struct`) composta da due variabili di tipo `double` per le coordinate x e y .

- **Griglia:** Array dinamico (`vector`) per la gestione della collezione di punti.

2.1.3 Complessità Attesa

La generazione della griglia richiede di iterare su tutti i punti del dominio discretizzato per un totale di $(N + 1)^2$ elementi. La complessità temporale attesa è dunque $O(N^2)$, poiché il calcolo delle coordinate di ciascun punto viene eseguito in tempo costante $O(1)$.

2.2 Sottomodulo 2: Gestore File

2.2.1 Specifiche e Funzionalità

Questo componente gestisce le operazioni di I/O su memoria secondaria, occupandosi sia della scrittura (esportazione) sia della lettura (importazione) delle informazioni strutturali e algebriche del problema.

- **Input:** Strutture dati in memoria (griglia, liste di nodi/archi, matrice sparsa A , vettore b) per le operazioni di scrittura; file di testo memorizzati per le operazioni di lettura.
- **Output:** File su memoria secondaria per la fase di esportazione; strutture dati caricate in memoria RAM per la fase di importazione.

Nello specifico, il sottomodulo mette a disposizione le funzionalità necessarie per:

- Generazione, scrittura e lettura del file contenente la lista dei soli punti interni della griglia (escludendo i punti di bordo).
- Generazione, scrittura e lettura del file contenente la lista degli archi che compongono il grafo di adiacenza.
- Generazione, scrittura e lettura del file contenente la lista dei nodi del grafo.

- Generazione, scrittura e lettura del file rappresentante la matrice sparsa dei coefficienti A .
- Generazione, scrittura e lettura del file contenente il vettore dei termini noti (rhs).

2.2.2 Strutture Dati

Per il Gestore File non si rendono necessarie strutture dati complesse aggiuntive in RAM. L'operazione richiede esclusivamente la gestione dei flussi di I/O (stream di input/output su file come `std::ifstream` e `std::ofstream`) per il trasferimento sequenziale dei dati tra la memoria primaria e i file di testo.

2.2.3 Complessità Attesa

Le operazioni di lettura e scrittura su memoria di massa richiedono il passaggio sequenziale attraverso i dati:

- Per i file topologici (nodi e archi): la complessità temporale attesa è $O(V + E) = O(N^2)$.
- Per i file algebrici (matrice A e vettore b): la complessità temporale attesa è proporzionale agli elementi non nulli della matrice e alla dimensione del vettore, ossia $O(|NZ| + |V|) = O(N^2)$.

2.3 Sottomodulo 3: Creazione del grafo di adiacenza

2.3.1 Specifiche e Funzionalità

Il componente si occupa della modellizzazione topologica dei nodi interni mediante la costruzione di un grafo non orientato.

- **Input:** Intero N che determina la discretizzazione uniforme del dominio quadrato $[0, 1]^2$.
- **Output:** Grafo non orientato $G = (V, E)$.

Il grafo modellato soddisfa le seguenti caratteristiche:

- Ogni nodo in V rappresenta un punto interno della griglia (escludendo i nodi di bordo).
- Ogni arco in E rappresenta una relazione di adiacenza orizzontale o verticale tra punti vicini (struttura a 5 punti).
- Ogni nodo possiede coordinate geometriche (x, y) associate.

Le funzionalità richieste per questo sottomodulo includono:

- Generazione dei soli punti interni della griglia.
- Assegnazione di identificatori numerici progressivi ai nodi.

- Costruzione delle relazioni di adiacenza tra nodi adiacenti.
- Esportazione dei dati nei file `coords.txt` e `connectivity.txt`.
- Supporto a successivi riordinamenti dei nodi (es. indicizzazione per algoritmi di fill-in reduction).

2.3.2 Strutture Dati

Per rappresentare la topologia del grafo in modo efficiente si suggeriscono:

- **Liste di adiacenza:** Implementate tramite `std::vector<std::vector<int>>` per garantire una rappresentazione compatta ed efficiente del grafo sparso.
- **Mappatura geometrica/topologica:** Strutture dati ausiliarie (come hash map o array bidimensionali di indicizzazione) per la conversione bidirezionale tra coordinate geometriche (i, j) e identificatori numerici progressivi dei nodi.

2.3.3 Complessità Attesa

La costruzione del grafo e la determinazione delle adiacenze richiedono una complessità temporale attesa pari a $O(|V|) = O((N - 1)^2) = O(N^2)$, poiché il grado di ciascun nodo è limitato superiormente da 4 e l'identificazione dei vicini viene eseguita in tempo costante.

2.4 Sottomodulo 4: Costruzione dell'ordinamento dei nodi

2.4.1 Specifiche e Funzionalità

Questo sottomodulo implementa la tecnica di riordinamento dei nodi del grafo per ridurre il fenomeno del *fill-in* durante la successiva fattorizzazione di Cholesky della matrice associata.

- **Input:** File `coords.txt` (coordinate dei nodi) e `connectivity.txt` (topologia degli archi del grafo).
- **Output:** File contenente la sequenza ordinata degli identificatori dei nodi (permutazione).

Le funzionalità richieste comprendono:

- Lettura ed elaborazione del grafo di adiacenza a partire dai file d'ingresso.
- Riordinamento dei nodi tramite una procedura ricorsiva basata su strategie di partizionamento (es. Dissecazione Annidata / *Nested Dissection*).
- Identificazione ricorsiva di separatori di vertici che dividono il grafo in sottografi disgiunti, numerando prima i sotto-problemi e infine il separatore.

- Scrittura della permutazione dei nodi ottenuta su file di output.

2.4.2 Strutture Dati

Per supportare la procedura ricorsiva di riordinamento si suggeriscono:

- **Permutazione dei nodi:** Un `std::vector<int>` per memorizzare l'ordine finale dei nodi.
- **Mappatura dei sottografi:** Strutture ausiliarie (come vettori di flag booleani `std::vector<bool>` o sottoliste di vertici) per tracciare i nodi attivi in ciascuna chiamata ricorsiva.
- **Coda / Pila ausiliaria:** Strutture dati di supporto (`std::queue` o `std::vector`) per la ricerca e l'individuazione del separatore di vertici (es. tramite viste BFS/RCM).

2.4.3 Complessità Attesa

La procedura ricorsiva di dissecazione divide il dominio ad ogni livello. La complessità temporale attesa dipende dal costo del partizionamento ad ogni livello di ricorsione:

- Il costo per trovare un separatore su un sottografo con V' nodi è proporzionale alle dimensioni del sottografo stesso, ossia $O(V')$.
- Risolvendo la relazione di ricorrenza $T(V') = 2T(V'/2) + O(V')$, si ottiene una complessità temporale complessiva attesa di $O(|V| \log |V|) = O(N^2 \log N)$.

2.5 Sottomodulo 5: Generazione della matrice sparsa

2.5.1 Specifiche e Funzionalità

Questo sottomodulo si occupa dell'assemblaggio del sistema lineare derivante dalla discretizzazione alle differenze finite dell'operatore laplaciano 2D sul dominio quadrato.

- **Input:** Grafo di adiacenza $G = (V, E)$ (o lista dei nodi interni con topologia dei vicini) e vettore di permutazione dei nodi (opzionale per l'ordinamento).
- **Output:** Matrice dei coefficienti A memorizzata in un formato efficiente per matrici sparse.

Le funzionalità richieste includono:

- Costruzione dello stencil a 5 punti per l'operatore laplaciano: elemento diagonale pari a 4, elementi fuori diagonale corrispondenti ai vicini adiacenti pari a -1 .

- Applicazione dell'eventuale permutazione dei nodi ricevuta in input per strutturare la matrice secondo l'ordinamento calcolato nel sottomodulo precedente.
- Rappresentazione della matrice in formato compresso per evitare l'allocazione inutilizzata di zeri.

2.5.2 Strutture Dati

Per rappresentare la matrice sparsa in modo compatto si suggeriscono:

- **Formato CSR (Compressed Sparse Row) o COO (Coordinate Format):** Vettori dinamici `std::vector<double>` per i valori dei coefficienti, `std::vector<int>` per gli indici di colonna e per i puntatori di riga.
- **Mappatura delle permutazioni:** Vettore `std::vector<int>` per tradurre l'indice originario del nodo nell'indice permutato.

2.5.3 Complessità Attesa

Essendo lo stencil limitato a un massimo di 5 elementi non nulli per riga, il numero totale di elementi non nulli nella matrice A è $|NZ| \leq 5|V|$. La complessità temporale attesa per la costruzione della matrice sparsa è quindi proporzionale al numero di nodi interni: $O(|V|) = O((N-1)^2) = O(N^2)$.

2.6 Sottomodulo 6: Generazione del termine noto

2.6.1 Specifiche e Funzionalità

Questo sottomodulo genera il vettore dei termini noti b (*rhs*) relativo all'equazione del calore in regime stazionario o al passo temporale della discretizzazione, integrando la sorgente di calore e le condizioni al contorno.

- **Input:** Griglia di punti interni, eventuale funzione sorgente $f(x, y)$, condizioni al contorno di Dirichlet/Neumann definite sul bordo e vettore di permutazione dei nodi.

- **Output:** Vettore dei termini noti b .

Le funzionalità richieste includono:

- Valutazione della funzione sorgente $f(x, y)$ in corrispondenza di ciascun punto interno della griglia.
- Modifica dei valori in prossimità del bordo per incorporare le condizioni al contorno note (es. traslazione al membro destro dei contributi dei nodi di bordo Dirichlet).
- Riordinamento delle componenti del vettore b in accordo con la permutazione dei nodi adottata per la matrice.

2.6.2 Strutture Dati

Per la memorizzazione e la manipolazione del termine noto si suggerisce:

- **Vettore denso:** Un array dinamico `std::vector<double>` di dimensione pari al numero di nodi interni $|V| = (N - 1)^2$.

2.6.3 Complessità Attesa

La valutazione della funzione sorgente e l'applicazione delle condizioni al bordo avvengono puntualmente su ogni nodo interno:

- La valutazione di ciascun elemento e l'eventuale reindicizzazione richiedono tempo costante $O(1)$ per nodo.
- La complessità temporale attesa complessiva è pari a $O(|V|) = O((N - 1)^2) = O(N^2)$.

2.7 Sottomodulo 7: Conversione CSC e Fattorizzazione di Cholesky

2.7.1 Specifiche e Funzionalità

Questo sottomodulo gestisce la conversione della matrice sparsa simmetrica e definita positiva nel formato compresso per colonne e l'esecuzione della fattorizzazione di Cholesky per la risoluzione del sistema lineare $Ax = b$ (oppure $\tilde{A}y = \tilde{b}$ con la matrice permutata).

- **Input:** Matrice sparsa A (in formato COO o CSR) e vettore dei termini noti b .
- **Output:** Matrice triangolare inferiore L della fattorizzazione di Cholesky ($A = LL^T$) in formato CSC e la soluzione del sistema x .

Le funzionalità richieste comprendono:

- Conversione efficiente della struttura della matrice sparsa nel formato CSC (*Compressed Sparse Column*), ottimale per gli algoritmi di fattorizzazione per colonna.
- Calcolo simbolico ed eliminazione per la fattorizzazione di Cholesky $A = LL^T$, sfruttando la simmetria e la struttura sparsa della matrice.
- Risoluzione del sistema tramite sostituzione in avanti ($Ly = b$) e sostituzione all'indietro ($L^Tx = y$).
- Valutazione dell'impatto del riordinamento dei nodi sulla riduzione del fenomeno di *fill-in* (confrontando il numero di elementi non nulli di L con e senza riordinamento).

2.7.2 Strutture Dati

Per rappresentare la matrice CSC e la fattorizzazione si utilizzano:

- **Struttura Matrice CSC:** Vettori dinamici `std::vector<double>` *values* per i valori non nulli, `std::vector<int>` *row_indices* per gli indici di riga, e `std::vector<int>` *col_pointers* per i puntatori d'inizio di ciascuna colonna (dimensione $n + 1$).
- **Fattore Cholesky L :** Una seconda struttura CSC dedicata alla memorizzazione della matrice triangolare inferiore risultante L .
- **Vettori di lavoro ausiliari:** Vettori densi `std::vector<double>` per memorizzare i risultati intermedi durante le fasi di eliminazione e di sostituzione avanti/indietro.

2.7.3 Complessità Attesa

L'analisi temporale si articola in due fasi distintive:

- **Conversione in CSC:** L'ordinamento e il raggruppamento per colonna dei $|NZ|$ elementi non nulli richiede tempo $O(|NZ| + |V|) = O(N^2)$.
- **Fattorizzazione di Cholesky:** Su una griglia 2D $N \times N$, la presenza di *fill-in* porta la fattorizzazione ad avere una complessità temporale di $O(N^3) = O(|V|^{3/2})$ in assenza o presenza di opportuno riordinamento (come la *Nested Dissection*), migliorando nettamente rispetto al caso denso $O(|V|^3) = O(N^6)$.
- **Risoluzione triangolare (Forward/Backward substitution):** Proporzionale al numero di elementi non nulli di L , pari a $O(|NZ(L)|) = O(N^2 \log N)$ con riordinamento ottimale.

2.8 Sottomodulo 8: Analisi sperimentale e confronto delle prestazioni

2.8.1 Specifiche e Funzionalità

L'ultimo sottomodulo si occupa dell'integrazione del flusso di calcolo all'interno di un Notebook interattivo per la valutazione delle prestazioni empiriche del solutore e la visualizzazione grafica dei risultati.

- **Input:** Sequenza di valori di discretizzazione $N = 32, 64, 128, 256, 512, 1024$.
- **Output:** Grafici delle prestazioni temporali e di memoria (*fill-in*), grafici della soluzione numerica $u(x, y)$ e mappe di radura (*sparsity patterns*) delle matrici.

Le funzionalità richieste comprendono:

- Esecuzione della risoluzione del sistema lineare per i differenti valori di N specificati.
- Confronto sistematico dei tempi di calcolo della fattorizzazione e del numero di entrate non-zero (NZ) richieste dal fattore L , valutati sia con l'ordinamento naturale (lessicografico) sia con il riordinamento proposto (*Nested Dissection*).
- Generazione dei plot della soluzione calcolata $u(x, y)$ sul dominio 2D nel Notebook.
- Visualizzazione grafica della struttura di radura della matrice dei coefficienti A e del fattore di Cholesky L (prima e dopo il riordinamento) mediante la funzione `matplotlib.pyplot.spy`.

2.8.2 Strutture Dati

Per l'analisi sperimentale si impiegano:

- **Array e DataFrame (Python/Jupyter):** Strutture dati tipo `numpy.ndarray` o `pandas.DataFrame` per la memorizzazione e organizzazione dei tempi di esecuzione, del conteggio dei non-zero (NZ) e degli errori residui.
- **Oggetti grafici Matplotlib:** Matrici e griglie di puntatori gestite da librerie esterne per la rappresentazione visiva dei dati.

2.8.3 Complessità Attesa

L'esecuzione dell'analisi benchmark per l'insieme dei valori $N \in \{32, \dots, 1024\}$ richiede:

- Una complessità temporale dominante determinata dal valore massimo $N_{max} = 1024$. La complessità complessiva delle simulazioni è $O(\sum N^3) = O(N_{max}^3)$.
- La generazione visiva dei pattern di radura tramite `spy` e la costruzione dei plot 2D/3D introducono un costo proporzionale unicamente al numero di elementi da tracciare, ovvero $O(|NZ(L)|)$ per ogni configurazione testata.